Advanced Computer Systems

Lecture 2

Dr. Shounak Chakraborty,
Assistant Professor,
Dept. of Computer Science,
Durham University, UK

Oct 20, 2026

Durham University

COMP3791

Course Outline (Adv. Comp. Arch.)

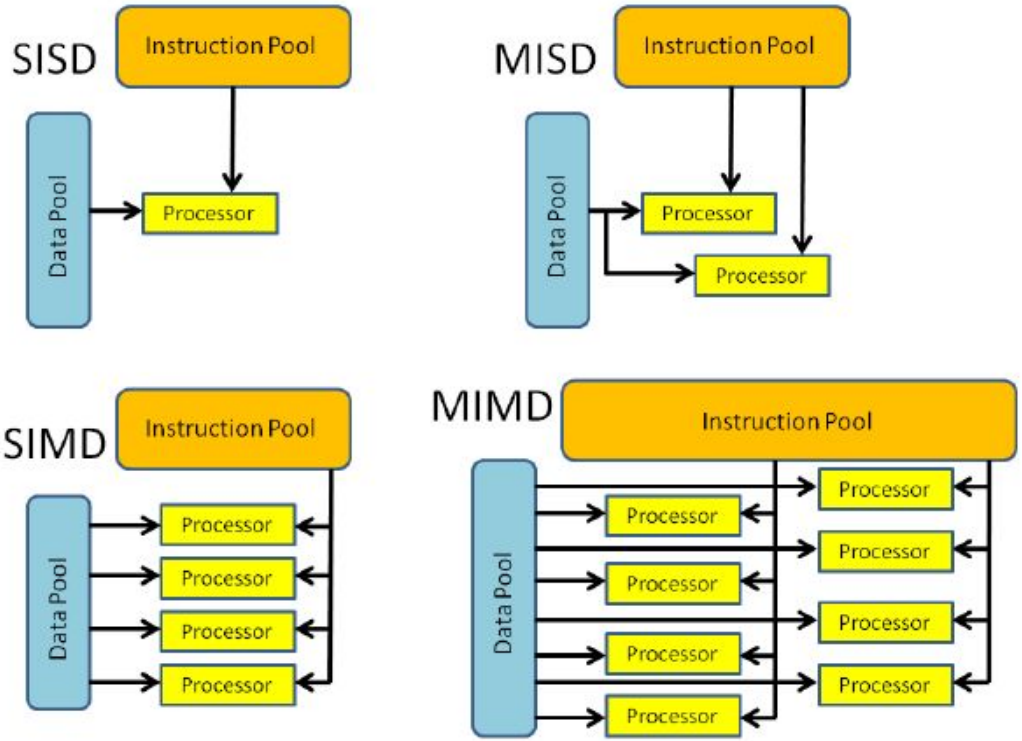
- **Introduction and Processor Fundamentals (Lec 5 hrs; WS 5 hrs)**
  - **Dennard Scaling, The Energy/Thermal Wall**
  - **ILP, Branches & BTB, Out-of-Order Execution**
  - **Data Parallelism, Vectors, Threads & Multi-cores**
  - **Load-Store Queues (LSQ), memory dependence speculation, store forwarding**
- **Caches, Memory and Optimisations (Lec. 5 hrs; WS 5 hrs)**
  - **Memory Hierarchies, Cache Designs, Write Policies**
  - **Cache Optimisations and 4Cs**

A Couple of Questions

- **When an OoO Core will experience a stall?**
  - Future instructions are directly or indirectly depends (true dependency) on the currently stalled instruction's outcomes
  - No more spare Registers to overcome WAW and WAR dependencies
  - ROB is filled up

- **Change in Core-Frequency–**
  - Change in IPC (Instructions Per Cycle)? ✗
  - Change in IPS (Instructions Per Second)? ✓

Flynn's Taxonomy



- Mike Flynn, "Very High-Speed Computing Systems," Proc. of IEEE, 1966

Data Parallelism and Vector Processors

- SISD: Single instruction operates on single data element
- SIMD: Single instruction operates on multiple data elements
  - Array processor
  - Vector processor
- MISD: Multiple instructions operate on single data element
  - Closest form: systolic array processor, streaming processor
- MIMD: Multiple instructions operate on multiple data elements (multiple instruction streams)
  - Multiprocessor
  - Multithreaded processor

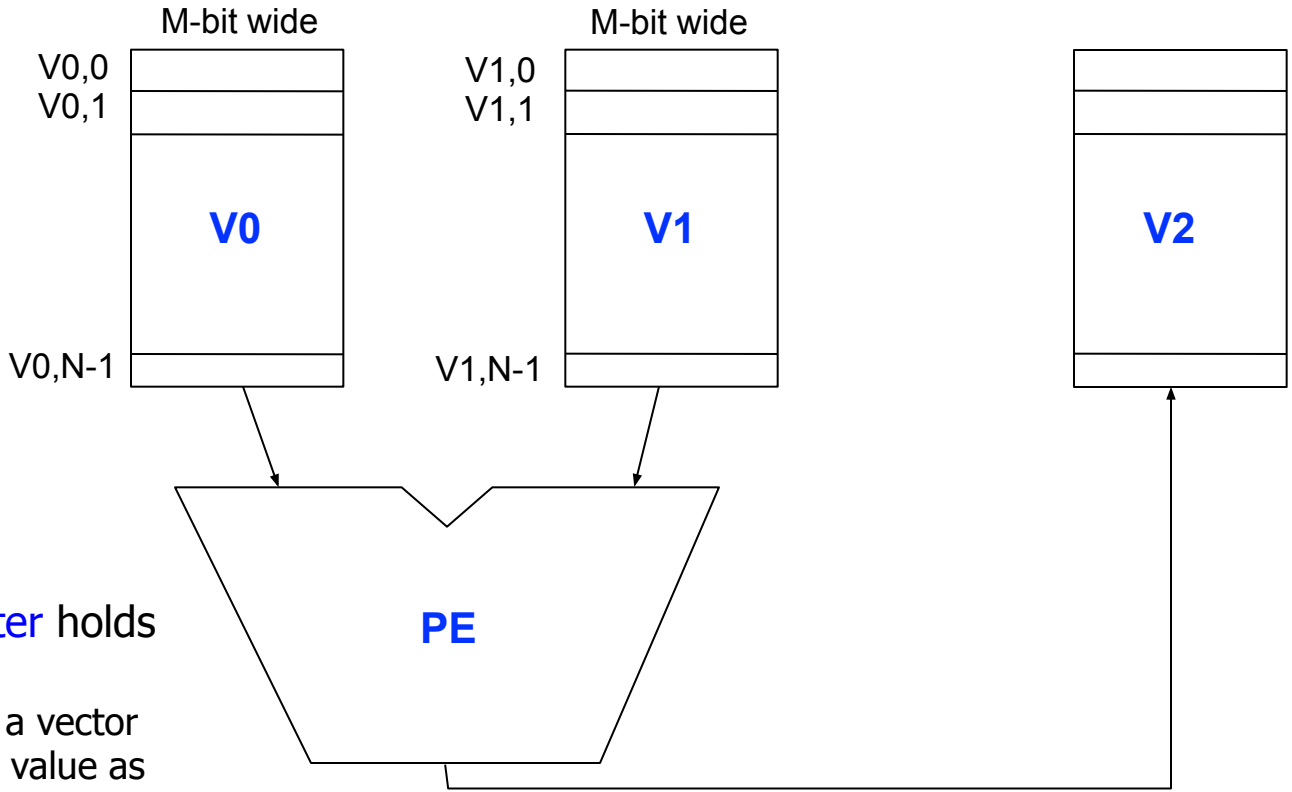
Data Parallelism

- Achieves by performing the same operation on different pieces of data
  - Single instruction multiple data (SIMD)
  - E.g., dot product of two vectors
- Contrast with data flow
  - Concurrency arises from executing different operations in parallel (in a data driven manner)
- Contrast with thread ("control") parallelism
  - Concurrency arises from executing different threads of control in parallel
- SIMD exploits operation-level parallelism on different data
  - Same operation concurrently applied to different pieces of data
  - A form of ILP where instruction happens to be the same across data

SIMD Processing Paradigm

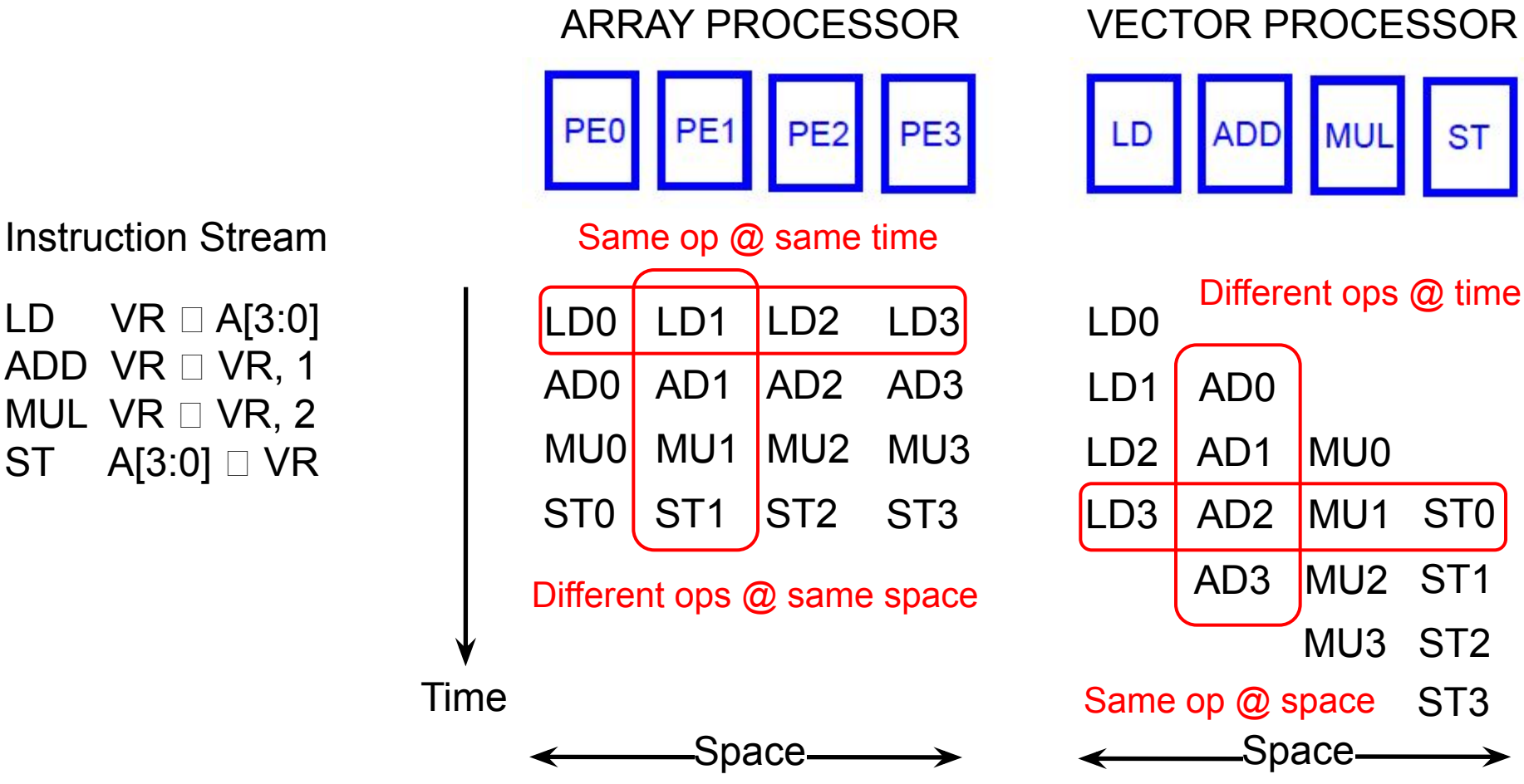
- Single instruction operates on multiple data elements
  - In time or in space
- Multiple processing elements (PEs), i.e., execution units
- Time-space duality
  - Array processor: Instruction operates on multiple data elements at the same time using different spaces (PEs)
  - Vector processor: Instruction operates on multiple data elements in consecutive time steps using the same space (PE)

Vector Registers: Multiple Data Storage



- Each vector data register holds N M-bit values
  - Each register stores a vector
  - Not a (single) scalar value as we saw before

Array vs. Vector Processor



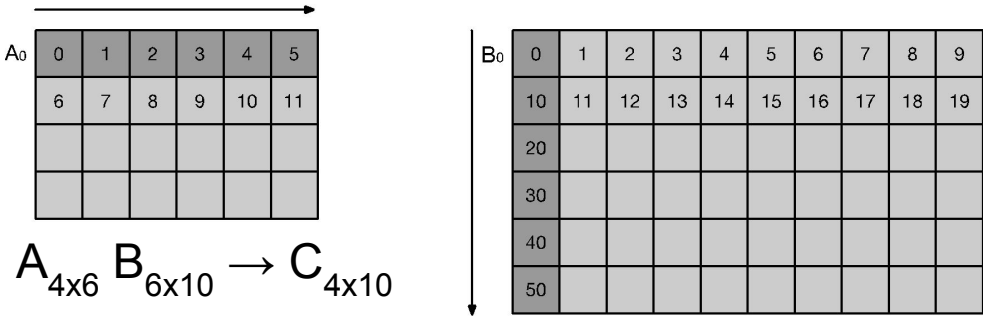
Vector Processor (I)

- A vector is a one-dimensional array of numbers
- Many scientific/commercial programs use vectors

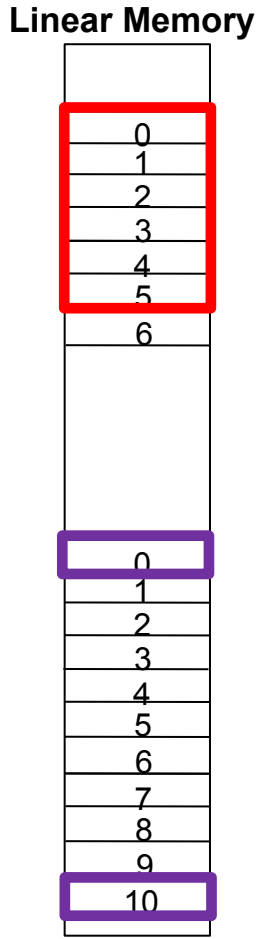
```
for (i = 0; i<=49; i++)
    C[i] = (A[i] + B[i]) / 2
```

- A vector processor is one whose instructions operate on vectors rather than scalar (single data) values
- Basic requirements
  - Need to load/store vectors ☐ vector registers (contain vectors)
  - Need to operate on vectors of different lengths ☐ vector length register (VLEN)
  - Elements of a vector might be stored apart from each other in memory ☐ vector stride register (VSTR)
    - Stride: distance in memory between two elements of a vector

Vector Stride Example: Matrix Multiplication



Dot product of each row vector of A with each column vector of B



- A and B matrices, both stored in memory in row-major order
- Load A's row 0 (A0 through A5) into vector register V_1
  - Each time, increment address by 1 to access the next column
  - Accesses have a stride of 1
- Load B's column 0 (B0 through B50) into vector register V_2
  - Each time, increment address by 10 to access the next row
  - Accesses have a stride of 10

Vector Processor (II)

- A vector instruction performs an operation on each element in consecutive cycles
  - Vector functional units are pipelined
  - Each pipeline stage operates on a different data element

- Vector instructions allow deeper pipelines
  - No intra-vector dependencies ☐ no hardware interlocking needed within a vector
  - No control flow within a vector
  - Known stride allows easy address calculation for all vector elements
    - Enables easy loading (or even early loading, i.e., prefetching) of vectors into registers/cache/memory

Vector Processors: Advantages

+ No dependencies within a vector
  - Pipelining & parallelisation work really well
  - Can have very deep pipelines (without the penalty of deep pipelines)

+ Each instruction generates a lot of work (i.e., operations)
  - Reduces instruction fetch bandwidth requirements
  - Amortizes instruction fetch and control overhead over many data

    --> Leads to high energy efficiency per operation

+ No need to explicitly code loops
  - Fewer branches in the instruction sequence

+ Highly regular memory access pattern

Vector Processors: Disadvantages and Limitations

-- Works (only) if parallelism is regular (data/SIMD parallelism)

++ Vector operations

-- Very inefficient if parallelism is irregular

-- How about searching for a key in a linked list?

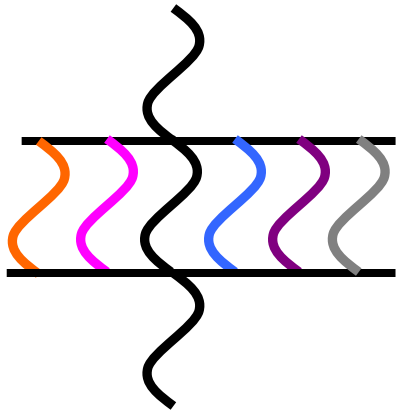
-- Memory (bandwidth) can easily become a bottleneck, especially if

1. compute/memory operation balance is not maintained
2. data is not mapped appropriately to memory banks

Amdahl's Law

- Amdahl's Law
  - f: Parallelisable fraction of a program
  - N: Number of processors

$$\text{Speedup} = \frac{1}{1 - f + \frac{f}{N}}$$



- Maximum speedup limited by serial portion: Serial bottleneck
- All parallel machines "suffer from" the serial bottleneck

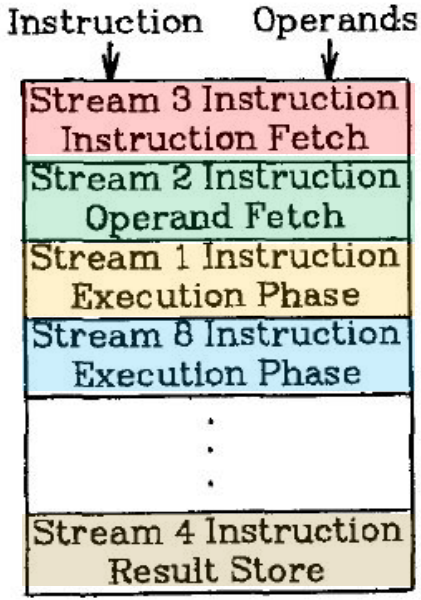
Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," AFIPS 1967.

Multithreading

- ILP boosts up performance but might not be good enough for some applications, e.g. online transaction/query processing, etc.
- Thread Level Parallelism (TLP) can be a promising approach
  - Each thread has its own data and set of instructions
  - Multiple Threads can be executed in parallel
- Multithreading allows multiple threads to share the functional units of a single processor in an overlapping fashion
  - Context of each thread must be stored in a separate copy of the register file, a separate PC, and a separate page table
  - Switching between threads can be an issue
- Fine Grained vs. Coarse Grained Multithreading

Fine-Grained Multithreading (cont..)

- Idea: Fetch from a different thread every cycle such that no two instructions from a thread are in the pipeline concurrently
  - Hardware has multiple thread contexts (PC+registers per thread)
  - Threads are completely independent
  - No instruction is fetched from the same thread until the prior branch/instruction from the thread completes



+ No logic needed for handling control and data dependencies within a thread
+ High thread-level throughput
-- Single thread performance suffers
-- Extra logic for keeping thread contexts
-- Throughput loss when there are not enough threads to keep the pipeline full

Fine-Grained Multithreading (cont..)

Each pipeline stage has an instruction from a different, completely-independent thread

We need a PC and a register file for each thread + muxes and control

- Tolerates control and data dependence resolution latencies by overlapping the latency with useful work from other threads
- Improves pipeline utilisation by taking advantage of multiple threads
- Improves thread-level throughput but sacrifices per-thread throughput and latency

Fine-Grained Multithreading (cont..)

- **Advantages**

  + No need for dependence checking between instructions
    (only one instruction in pipeline from a single thread)
  + No need for branch prediction logic
  + Otherwise-bubble cycles used for executing useful instructions from different threads
  + Improved system throughput, latency tolerance, pipeline utilisation

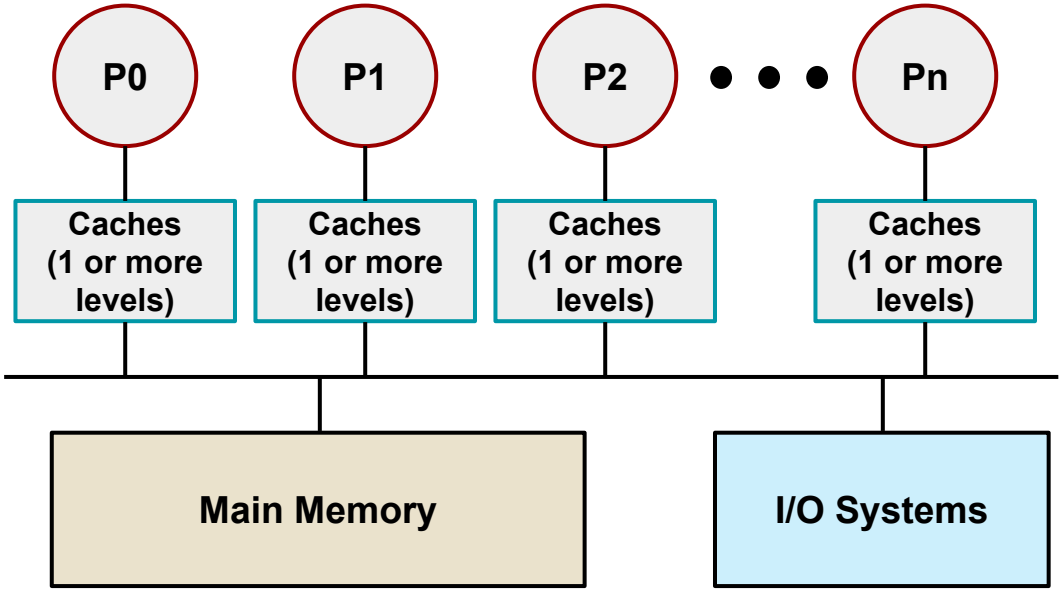
- **Disadvantages**

  - Extra hardware complexity: multiple hardware contexts (PCs, register files, …), thread selection logic
  - Reduced single thread performance (one instruction fetched every N cycles from the same thread)
  - Resource contention between threads in caches and memory
  - Dependence checking logic *between* threads may be needed (load/store)

Coarse-Grained Multithreading

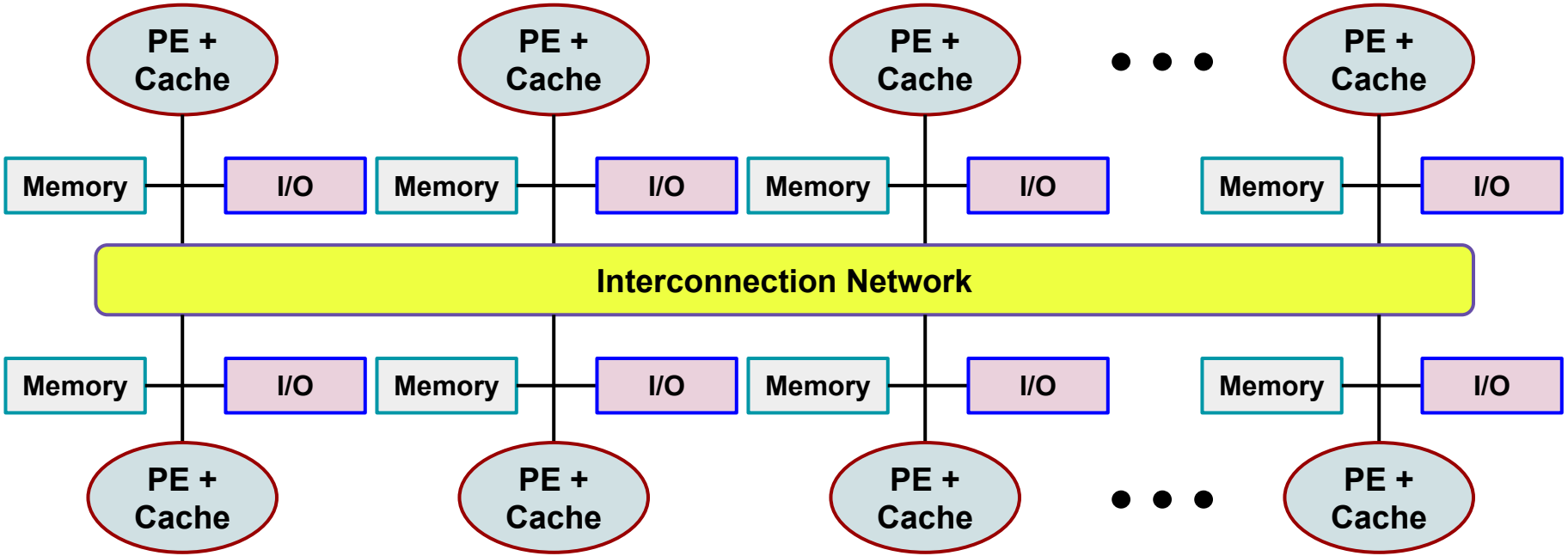
- Coarse-grained multithreading: an alternative to fine-grained Multithreading
- Coarse-grained multithreading switches threads only on costly stalls, such as level 2 cache misses, i.e. during costly stalls
- Downsides:
  - limited in its ability to overcome throughput losses, especially from shorter stalls
  - pipeline start-up costs of coarse-grain multithreading
    - when a stall occurs, the pipeline must be emptied or frozen
    - new thread that begins executing after the stall must fill the pipeline before instructions will be able to complete

MIMD and Chip Multi-Processors



- Concept of a centralised shared-memory (CSM) multiprocessor
- Multiple processor-cache subsystems share the same physical memory, typically connected by one or more buses or a switch
- The key architectural property is the uniform access time to all of memory from all the processors

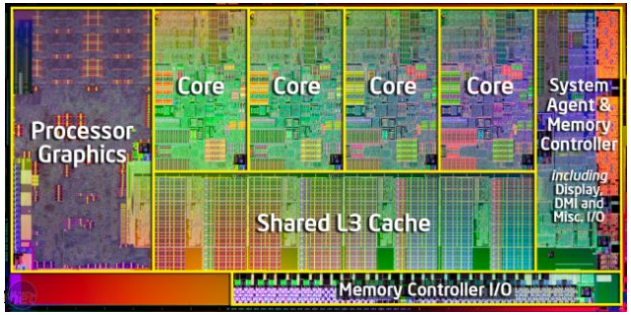
MIMD and Chip Multi-Processors (cont…)

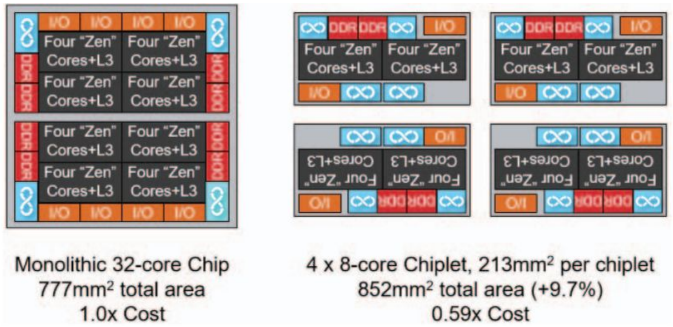


- Basic architecture of a distributed shared memory (DSM) multiprocessor consists of individual nodes containing a processor, some memory, typically some I/O, and an interface to an interconnection network that connects all the nodes
- Individual nodes may contain a small number of processors, which may be interconnected by a small bus or a different interconnection technology, which is less scalable than the global interconnection network

MIMD and Chip Multi-Processors (cont…)

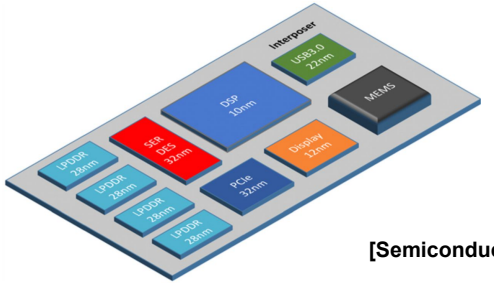
Example: A Modern CMP [Intel]





[AMD]

- Single chip has more than one PEs
  - May be 1000 PEs
- Multi-levels of caches are on-chip
  - Two levels of local caches
- LLCs are usually shared
  - Coherence is an issue

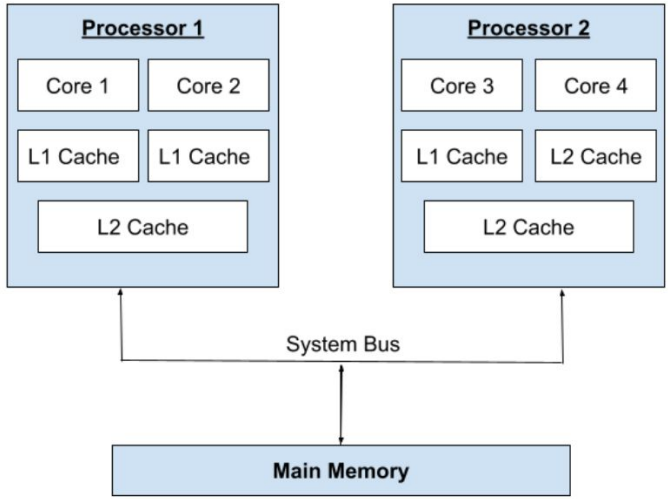


[Semiconductor Engineering]

Homogeneous and Heterogeneous CMPs

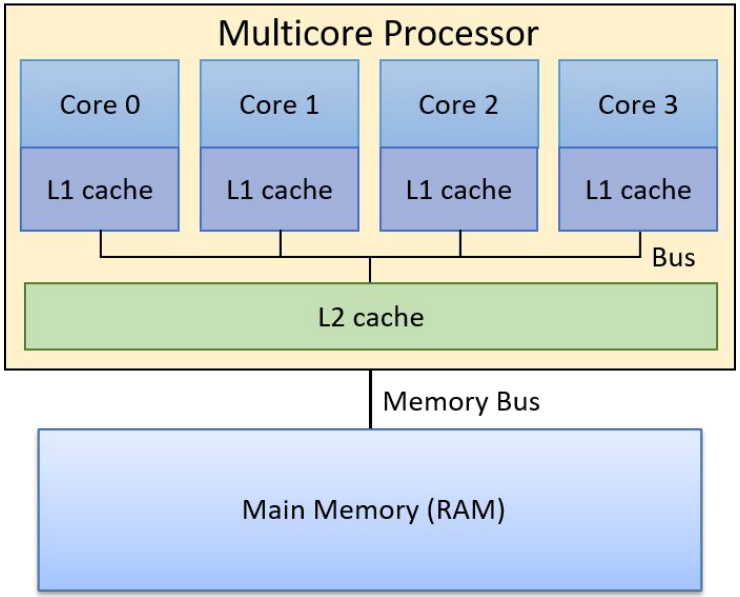
Foundations of Homogeneous CMPs

- **SMP on a Single Die:** Replicating identical, general-purpose cores across a unified silicon substrate to exploit Thread-Level Parallelism (TLP)
- **Uniform Microarchitecture:** Every core shares the exact same ISA, pipeline depth, execution resources, and branch predictor designs
- **Shared Clock & Voltage Domain:** Cores operate off a synchronised clock and power plane, significantly simplifying board-level power supply design
- **Overcoming Frequency Walls:** Successfully transitioned compute scaling from unsustainable GHz frequency pushes to horizontal thread throughput scaling



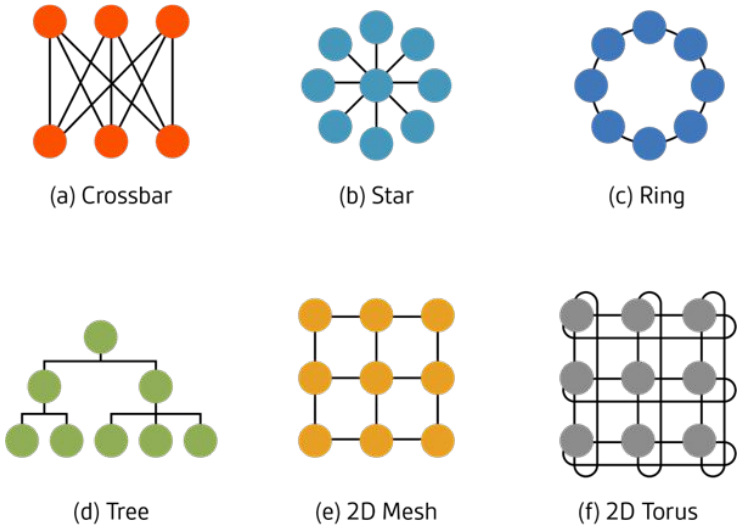
Shared-Memory & Cache Topologies

- **Private L1 / L2 Caches:** Dedicated low-latency instruction and data caches provide per-core bandwidth isolation and minimize memory stall penalties
- **Shared Last-Level Cache (LLC / L3):** Dynamically pooled on-chip cache slices enable zero-copy inter-thread data sharing across all active physical cores
- **UCA vs. NUCA Latency:** Uniform Cache Access (UCA) offers identical latency for low core counts; Non-Uniform Cache Access (NUCA) partitions slices based on physical die proximity
- **Cache Coherence Overhead:** Enforces global memory consistency using hardware protocols (MESI/MOESI) across private and shared lines



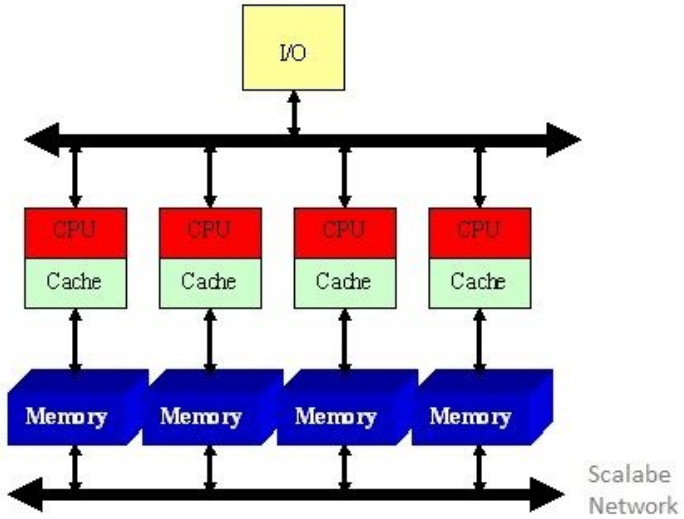
On-Chip Interconnect Fabrics

- **Shared Bus:** Central broadcast medium; straightforward arbitration and snooping, but saturates rapidly beyond 4–8 cores
- **Ring Interconnects:** Bidirectional pipelined rings linking core stations with bounded, deterministic routing delays (standard in 8–12 core dies)
- **Crossbar Switches:** Non-blocking point-to-point crossbars offer zero contention, but area scales quadratically ($O(N^2)$)
- **2D Mesh NoCs:** Packet-switched Network-on-Chip grid routers deliver terabyte/sec bisection bandwidth for high-core server silicon



Memory Architecture: UMA vs NUMA

- **Uniform Memory Access (UMA):** Cores access physical system RAM via a shared central memory controller with identical latency across all processing nodes
- **UMA Bottlenecks:** Multiple high-frequency homogeneous cores competing for a central memory bus trigger severe pin and interface saturation
- **Non-Uniform Memory Access (NUMA):** Physical memory is partitioned across sockets/chiplets; local node accesses yield high bandwidth and low latency
- **Interconnect Transit Penalties:** Remote memory requests must traverse inter-socket links (UPI, Infinity Fabric), introducing non-uniform latency hops



OS Scheduling in Homogeneous CMP

Symmetric Run-queue Balancing

- **Fungible Compute Units:** The OS scheduler treats all cores as computationally interchangeable, dispatching ready threads to any idle execution pipe
- **Work-Stealing Paradigms:** Underutilised cores inspect overloaded sibling run-queues and steal runnable tasks to maximise overall chip utilisation

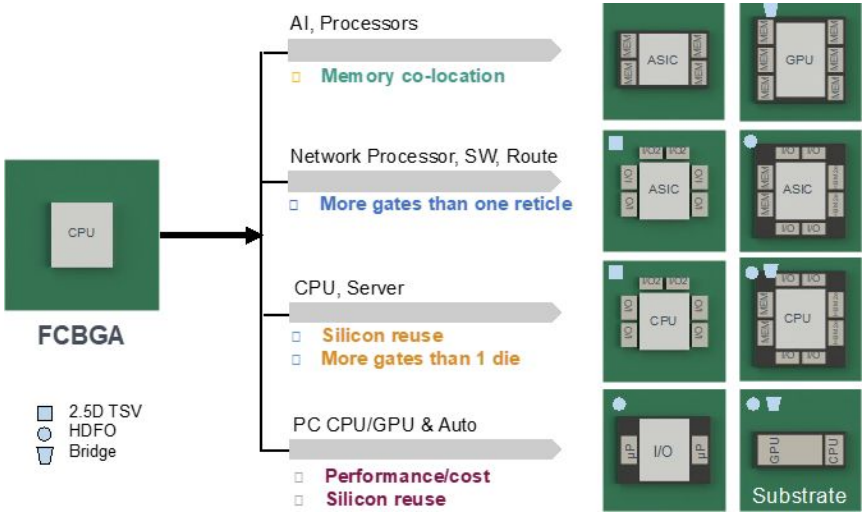
Affinity & Locality Overhead

- **Cache Warmth Preservation:** Schedulers pin active threads to specific cores to avoid cold misses in private L1/L2 caches
- **Synchronisation Bottlenecks:** Centralised run-queue locks introduce severe lock contention as homogeneous core counts scale to 64+ cores

Monolithic Dies vs Chiplets

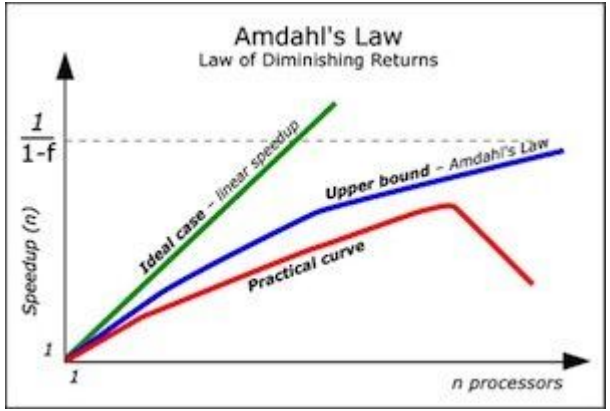
The Modular Packaging Shift

- Monolithic 64-core dies exceed 700mm², resulting in severe wafer defect rates and prohibitive manufacturing costs
- Modern symmetric architectures split execution compute into modular Core Complex Dies (CCDs) bound across dense silicon bridges and substrate packaging



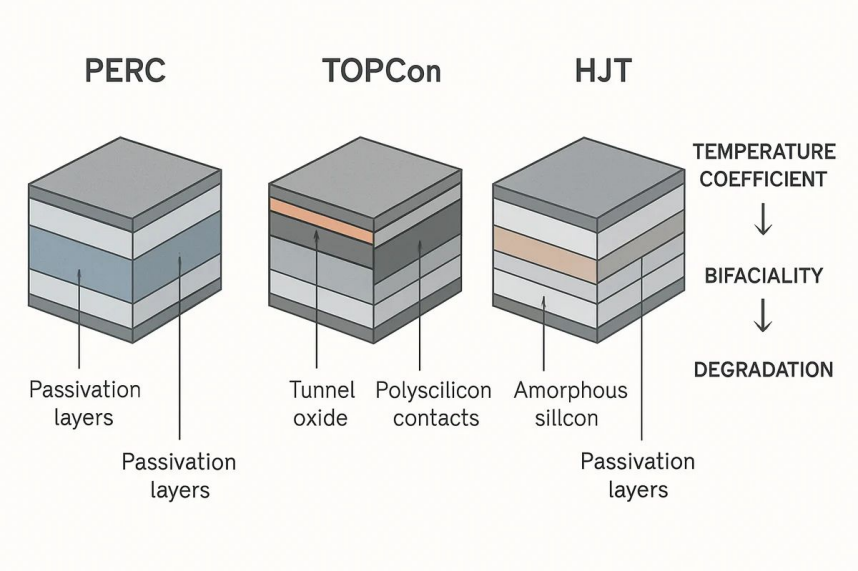
Amdahl's Law in Symmetric CMPs

- Theoretical speedup with parallel fraction *f* and *N* symmetric cores: $\text{Speedup} = 1/((1-f)+f_N)$
- **Serial Bottleneck:** As $N \to \infty$, speedup asymptotically caps at *1 / (1 - f)*, regardless of how many cores are deployed
- **Serial Phase Idleness:** During single-threaded execution phases, *N - 1* high-power cores sit completely idle, leaking static power



Limits of Scaling & Dark Silicon

- **Breakdown of Dennard Scaling:** Supply voltage scaling stalled while transistor counts kept climbing, causing power density to surge
- **Thermal Design Power (TDP) Cap:** Firing all symmetric cores at peak turbo frequencies exceeds package heat dissipation thresholds
- **The Dark Silicon Reality:** Up to 50%+ of a multi-core die must remain powered off or drastically underclocked to avoid thermal failure
- **The Asymmetric Solution:** Replaced uniform power-hungry cores with specialised asymmetric compute units and hardware accelerators



Intro to Heterogeneous Computing

- **Asymmetric Multiprocessing (AMP):** Integrates cores with deliberately distinct performance, area, and power envelopes on the same silicon
- **Unified Binary ISA:** All processing cores share identical instruction set architectures (e.g., ARM64, x86-64), enabling seamless code migration
- **Microarchitectural Contrast:** Blends wide, deep speculative out-of-order execution cores with compact, high-efficiency in-order/narrow cores
- **Dynamic Phase Matching:** Matches execution hardware complexity directly to instantaneous application requirements



Heterogeneous Silicon Integration: Pairing Wide and Dense Cores

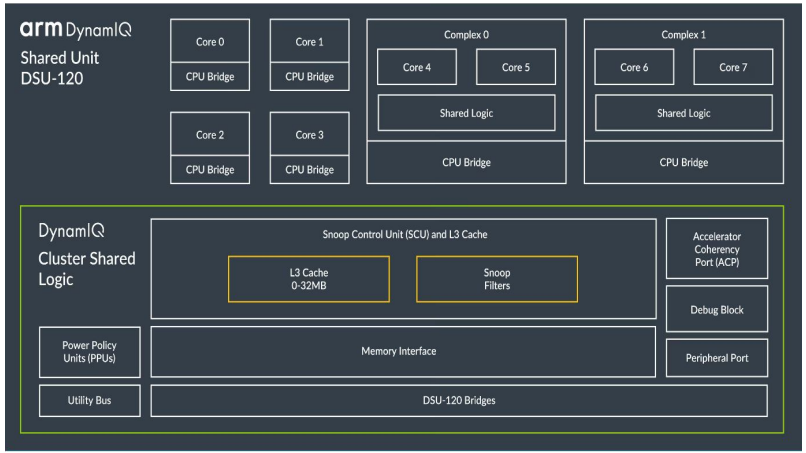
The Hill & Marty Asymmetric Model

- **Resource Budgeting (ISCA 2008)**
  - Assume a chip budget of *B* Base Core Equivalents (BCEs); an enhanced core consumes *r* resources to produce performance: perf(r)=r
  - Governed by Pollack's Rule: single-thread performance grows with the square root of microarchitectural area

- **Asymmetric Superiority**
  - **Accelerating the Serial Tail:** Dedicates one large core (*r > 1*) to accelerate the sequential execution phase *(1 - f)*
  - **Maximising Throughput:** Deploys the remaining *B - r* budget as small, energy-dense cores (*r = 1*) for the parallel phase *f*
  - **Theoretical Proof:** Asymmetric allocations consistently out-perform symmetric chips across broad workload distributions

Arm big.LITTLE & DynamIQ

- **"big" Complex (Cortex-X / A7xx):** Deep speculative Out-of-Order execution core with wide dispatch windows; optimized for burst latency and interactive tasks
- **"LITTLE" Complex (Cortex-A5xx):** Highly efficient shallow pipeline prioritizing Energy-Delay Product (EDP); executes background daemons and media playback
- **DynamIQ Shared Unit (DSU):** Eliminates cluster-level partitioning by housing big and LITTLE cores in a single coherent cluster sharing a unified L3 cache
- **Zero-Copy Task Migration:** Slashes thread migration latency from microsecond socket hops down to nanosecond shared-L3 transfers



Intel Hybrid x86 Architecture

- **Performance Cores (P-Cores):** Deeply pipelined, wide multi-issue out-of-order architecture with Hyper-Threading (SMT) and dedicated private L2 caches
- **Efficient Cores (E-Cores):** Four-core clusters sharing a high-density L2 cache; stripped of SMT to maximise multi-threaded silicon area efficiency
- **Unified Ring Bus:** P-cores and E-core 4-clusters interface directly to a shared Ring Interconnect tapping into a single shared LLC
- **ISA Parity Enforcement:** Disables divergent vector extensions (e.g., AVX-512) via hardware fuses to guarantee universal binary execution



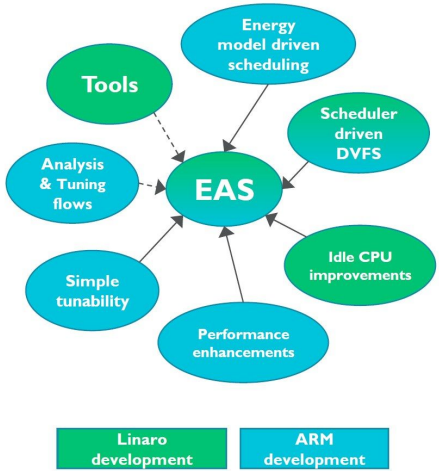
Intel Thread Director Telemetry

- **Embedded Hardware Telemetry:** Microcontroller-level sensors continuously monitor instruction streams (scalar INT, AVX2, branches, and memory stalls)
- **Dynamic Execution Classification:** Analyzes instructions in real time, assigning threads to execution classes based on compute vs. stall profiles
- **Enhanced Hardware Feedback Interface (EHFI):** Passes dynamic energy and performance capability tables directly to the OS scheduler
- **Intelligent Steering:** Directs compute-heavy vector loops to P-cores while shunting polling and background loops to E-cores



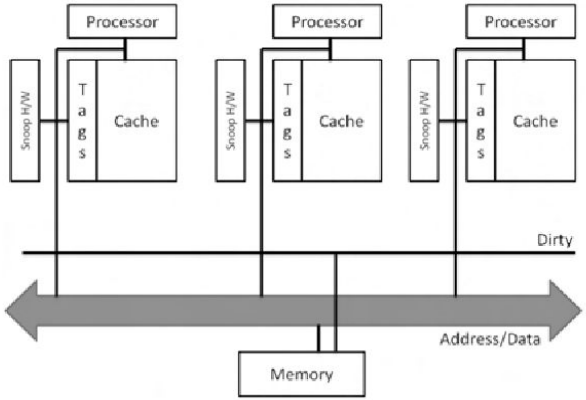
Intel Thread Director Telemetry

- **Active Energy Model (EM):** Maintains a calibrated power consumption table across all DVFS Operating Performance Points (OPPs) for each core type
- **Predictive Energy Evaluation:** Schedulers compute the marginal energy delta of placing a task on a LITTLE core vs. waking a big core
- **Capacity-Aware Placement:** Tasks are assigned to the most energy-frugal core capable of meeting the application's completion deadline
- **Hysteresis Thresholding:** Prevents excessive ping-pong core migrations, which incur heavy context save/restore and cold cache penalties



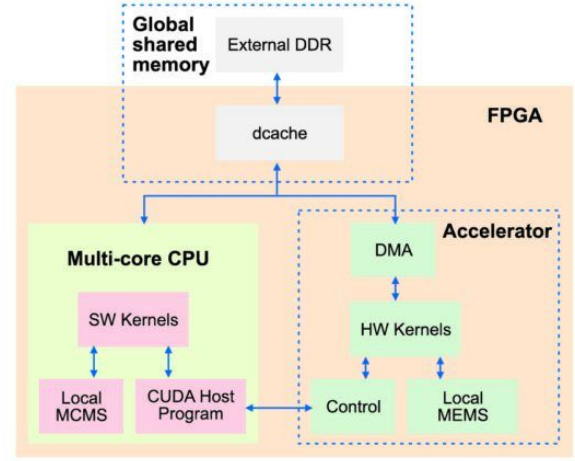
Heterogeneous Cache Coherence

- **Asymmetric Snooping Loads:** High-throughput big cores generate heavy broadcast traffic that can overwhelm narrower snoop ports on little cores
- **Hierarchical Snoop Filtering:** Centralised directory tags filter out non-shared coherence inquiries, shielding efficient cores from unnecessary stalls
- **Architectural Memory Consistency:** Both core types must strictly adhere to the same memory ordering model (e.g., TSO on x86, Weak Ordering on ARM)
- **Unified Store Ordering:** Speculative load forwarding and store queues reconcile cross-core atomic locks transparently without race conditions



The Heterogeneous Compute Future

- **CPU Asymmetry Standardised:** Heterogeneous core pairings are now ubiquitous across mobile, client desktop, and cloud server platforms
- **System-on-Chip (SoC) Acceleration:** Modern silicon pairs asymmetric CPUs with specialized accelerators (NPUs, GPUs, ISPs, DSPs) to bypass dark silicon walls
- **Advanced 2.5D/3D Packaging:** High-density micro-bumps and silicon interposers allow mixing different process nodes inside a single compact module
- **Hardware-OS Co-Design:** The future of compute efficiency relies on deep co-design between hardware telemetry engines and intelligent OS schedulers



Modern SoC Integration: CPUs, GPUs, NPUs, and Memory Subsystem

Summary

- Data Parallelism
- Vector vs. Array Processors
  - Time-space duality
- Multithreading
  - Fine-Grained vs. Coarse-Grained
- MIMD and Multi-cores
  - Multiple PEs in a single chip
    - Homogeneous and Heterogeneous

Thank
You